

SY31 – TP #5

Détection d'Objets à la Caméra

P26

1 Introduction

Les caméras sont des capteurs de perception de plus en plus présents car elles ont l'avantage de ne pas coûter cher et de fournir une information très dense. Cette quantité d'information vient néanmoins au prix d'un besoin important en ressources de calcul qui n'est disponible que depuis une quinzaine d'années. L'objectif de ce TP est d'utiliser la caméra embarquée sur le TurtleBot afin de détecter des objets. Pour cela, la bibliothèque OpenCV qui inclue de nombreux algorithmes de traitement d'images et de vision sera utilisée.

2 Calibration

L'image fournie par une camera présente généralement des déformations

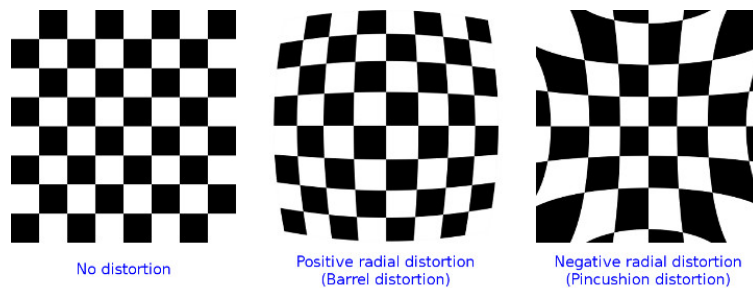


Figure 1: Types de déformation radiale

Le premier objectif de ce TP est donc de corriger ces déformations. Pour faire cela, une méthode classique consiste à photographier une grille composée de carrés blancs et noirs facilement détectables et aux coordonnées connues. Un problème de minimisation peut alors être résolu pour trouver les paramètres qui modélisent au mieux la caméra.

Plutôt que de réaliser ce travail nous même, nous allons utiliser un nœud ROS qui automatise ce processus¹.

¹Pour des applications non ROS, se référer au [tutoriel calibration](#) du site d'OpenCV.

Exercice 1 : En vous inspirant du TP1, créer un paquet `tp5` dans votre workspace et ajouter les nœuds `detect` et `project` donnés sur Moodle.

Exercice 2 : Ajouter le launch fourni sur Moodle au paquet.

Un launch est un fichier `xml` décrivant des nœuds à lancer en une seule commande. Pour l'ajouter :

- Créer un dossier `launch` à la racine du paquet
- Ajouter `camera.launch.xml` dedans
- Ajouter au `setup.py`
 - `from glob import glob` en haut du fichier
 - `('share/' + package_name + '/launch', glob('launch/*'))` dans les `data_files`

Exercice 3 : Démarrer le launch:

```
ros2 launch tp5 camera.launch.xml
```

Vérifier que l'image est bien reçue sur le topic `/turtlecam/image_raw`

Exercice 4 : Calibrer la caméra avec

```
ros2 run camera_calibration cameracalibrator --size 7x5 --square 0.033 \
--no-service-check --camera turtlecam --ros-args -r image:=/turtlecam/image_raw \
-r __ns:=/ -r /camera/set_camera_info:=/turtlecam/set_camera_info
```

Déplacer la [mire fournie](#) devant la caméra afin qu'elle soit détectée sous différents angles Varier les poses jusqu'à ce que les quatre paramètres de droite (position dans l'image, inclinaison et proximité de la mire) soient validés. Cliquer sur `calibrate` puis sur `commit` lorsque le résultat est satisfaisant

Éteindre et rallumer le robot pour que la calibration soit bien effective. Enfin, visualiser l'image rectifiée sur le topic `/turtlecam/image_rect`. Si le topic est vide, la calibration n'a pas marché et doit être refaite.

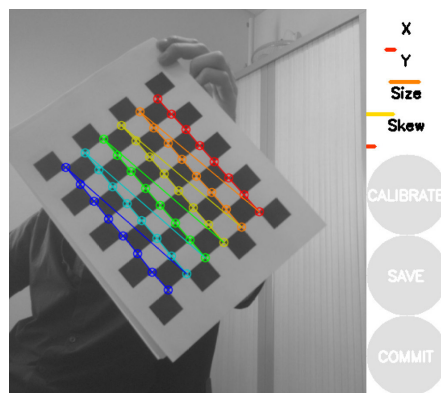


Figure 2: Exemple de calibration

En faisant cela, nous venons de déterminer les paramètres de distorsion D , la matrice intrinsèque K et la matrice de projection P . Ces informations sont désormais disponibles sur le topic `/turtlecam/camera_info` dans un message de type `sensor_msgs/CameraInfo`. On ne s'en servira pas directement mais ROS l'utilise pour rectifier l'image de `/turtlecam/image_rect`.

3 Détection d'Objet

Le second objectif du TP est de détecter un objet à l'aide de sa couleur. Nous nous restreindrons à des objets de couleur uniforme et facilement discernable du reste de l'image.

Exercice 5 : Déterminer un objet de couleur unie ainsi que sa couleur RGB. Pour cela, on pourra utiliser la figure suivante ou plus généralement, capturer l'image dans `rqt` et utiliser l'outil pipette d'un logiciel de type `Inkscape` ou `GIMP`.

Warning

Par défaut, GIMP donne des valeurs en pourcents, pensez bien à les demander entre 0 et 255.

Exercice 6 : Récupérer sur Moodle le modèle de nœud `detect.py`. Dans un premier temps, le but du nœud sera de détecter la couleur dans l'image :

- Déterminer un intervalle $[C_{min}; C_{max}]$ qui contient les couleurs désirées
- Dans la fonction `detect` utiliser ces intervalles en conjonction avec la fonction `inRange` d'OpenCV afin de produire un masque² des couleurs comprises dans l'intervalle
- Publier ce masque et le visualiser dans `rqt`

Warning

OpenCV travaille par défaut dans l'espace de couleur BGR et non le classique RGB.

Warning

La documentation d'OpenCV est donnée en C++ et en Python, pensez bien à regarder la signature Python !

Exercice 7 : Dans un second temps, on va chercher à détecter la position et la forme de l'objet :

- Utiliser la fonction `findContours` d'OpenCV sur le masque. La fonction retourne une polyligne décrivant le contour de chaque "objet" trouvé. Afficher les contours détectés sur l'image à en utilisant `drawContours` puis en publiant l'image
- Utiliser la fonction `contourArea` pour déterminer la taille des objets détectés. Ne garder que le plus gros objet et seulement s'il est suffisamment grand.
- Utiliser le [tutoriel suivant](#) pour dessiner la forme englobante adaptée autour du contour.

²Matrice booléenne

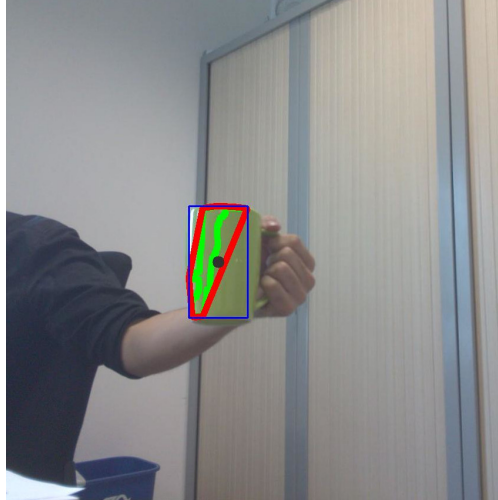


Figure 3: Exemple de détection

Exercice 8 : Améliorer les résultats en détectant la couleur dans l'espace HSV, qui est plus invariant aux conditions d'éclairage. Pour cela, `cv2.cvtColor(img, cv2.COLOR_BGR2HSV)` peut être utilisée. Attention à changer la façon de faire les intervalles, d'autant plus qu'OpenCV représente le HSV avec des valeurs dans 0;180 pour la teinte et 0;255 pour la saturation (contrairement aux 0;360 et 0;100 habituels).

i Note

Le modèle HSV, pour **Teinte** (Hue), **Saturation**, et **Valeur** (Value), est utilisé pour aligner plus étroitement la sélection de couleur avec la perception humaine, par rapport au modèle RGB.

- **Teinte (Hue):** représente la couleur sur la roue des couleurs, mesurée en degrés de 0 à 360 - de 0 à 180 dans OpenCV.
- **Saturation:** décrit l'intensité de la couleur, de totalement grise (0%) à la plus intense (100%) - de 0 à 255 dans OpenCV.
- **Valeur (Value):** indique la luminosité de la couleur, de noire (0%) à la plus lumineuse (100%) - de 0 à 255 dans OpenCV.

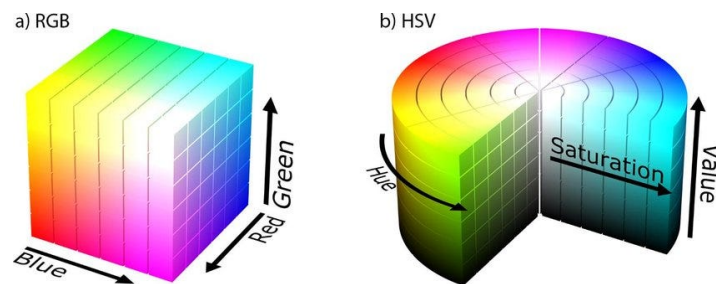


Figure 4: Roue HSV

Exercice 9 : Améliorer encore plus les résultats en utilisant des opérations [opérations mor-](#)

[phologiques](#). Celles-ci permettent de *boucher les trous* et retirer les pixels solitaires de la détection.

Finalement, on pourra simplifier la forme résultante en ajustant [une enveloppe](#) ou un [rectangle](#) autour.

4 Bonus : Projection d'un point dans l'image

La calibration réalisée dans la partie Section 2 nous a fourni la matrice de projection P . On peut l'utiliser pour projeter un point 3D (X, Y, Z) (par rapport à la caméra) dans l'image (x, y) :

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} = P * \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}, \quad x = u/w, \quad y = v/w$$

Exercice 10 : Récupérer sur Moodle le noeud `project.py`

- Remplir la fonction `callback_info` afin de récupérer la matrice P dans le message [sensor_msgs/msg/CameraInfo](#)
- Transformer P en une matrice `numpy` de taille 3×4
- Utiliser P pour projeter un point arbitraire dans l'image dans la fonction `callback`, l'afficher avec `drawCircle` et publier l'image résultante

Exercice 11 : Utiliser la mire comme point 3D ([tuto OpenCV](#))

- Récupérer les matrices K et D dans `callback_info` et les transformer en matrices 3×3 et 5×1
- Convertir l'image en niveaux de gris avec `cv.cvtColor(img, cv.COLOR_BGR2GRAY)`
- Détecter la mire à l'aide de [findChessboardCorners](#)
- Utiliser la fonction [solvePnP](#) pour déterminer la position de la mire dans l'espace 3D. Le premier argument décrit la grille en 3D et est fourni, les 3ème et 4ème arguments sont respectivement K et D . La fonction retourne les vecteurs de rotation et de translation entre la caméra et l'origine de la mire
- Utiliser ces vecteurs pour construire une matrice de transformation T de taille 3×4
- Faire la transformation $P * T * [X, Y, Z, 1]'$ pour projeter un point arbitraire dans l'image par rapport à la mire
- Projeter 4 points correspondant au repère de la mire (origine, axe x , axe y et axe z)
- Les afficher avec la fonction `drawAxes`, publier l'image et l'afficher dans `rqt`

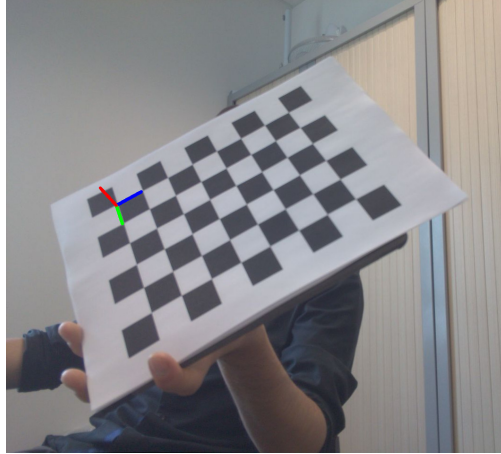


Figure 5: Exemple de projection